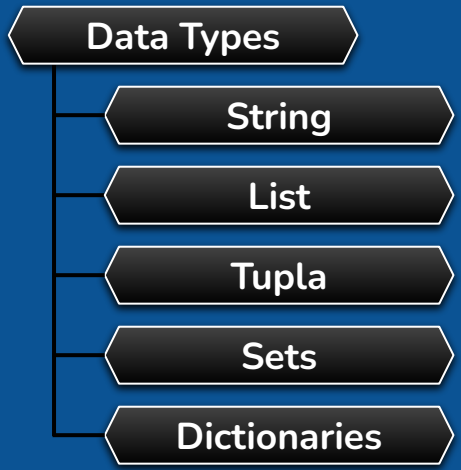
Desenvolupament web en entorn servidor

PYTHON

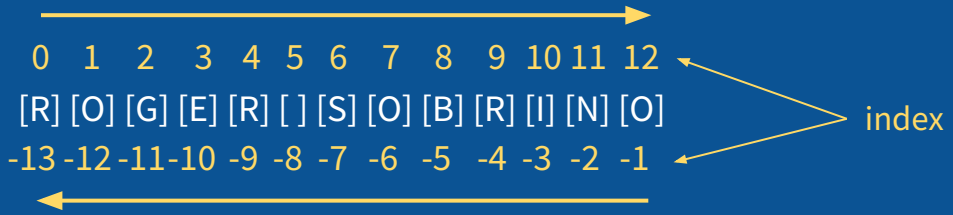
RA's vinculades:
RA3: Escriu blocs de sentències embeguts en llenguatges de marques, seleccionant i utilitzant les estructures de programació.

PYTHON

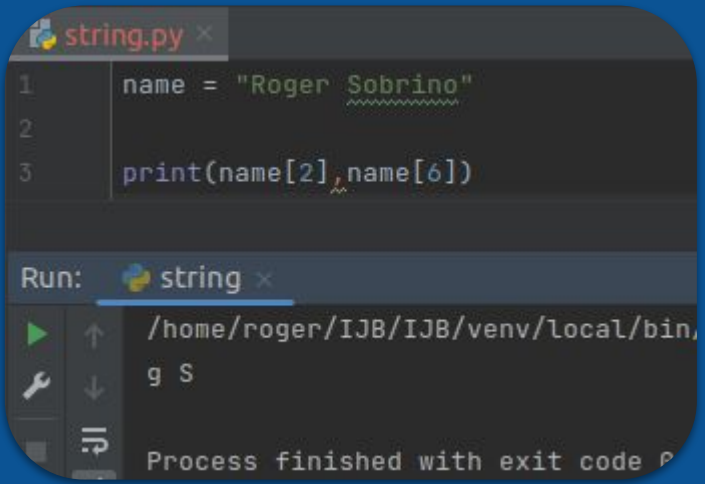
INDEX

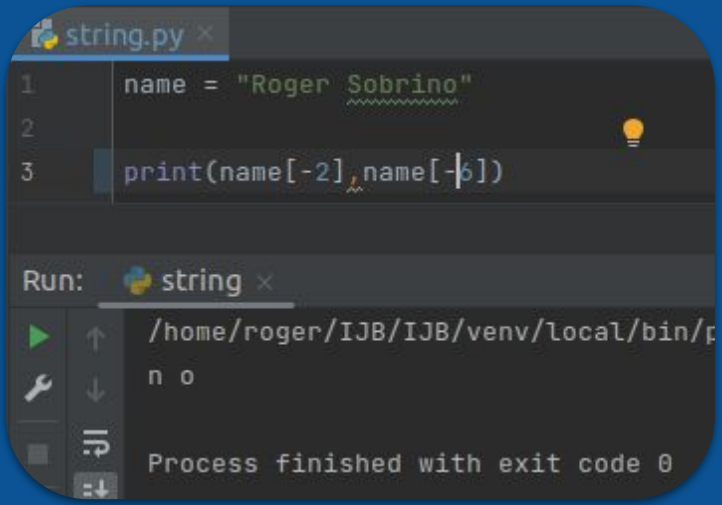


Data Types | String

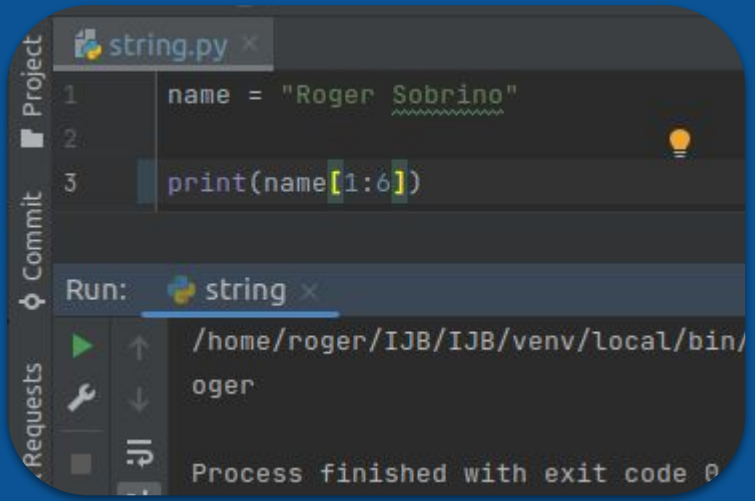


- Extreure caràcters d'un String





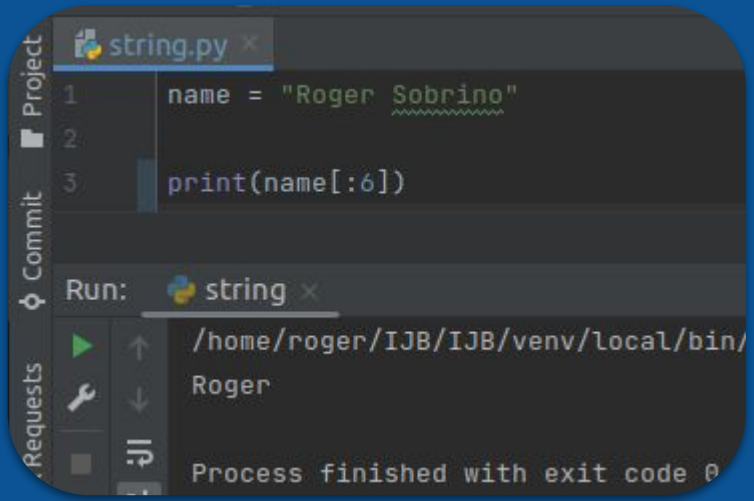
- Extreure caràcters d'un String



0	1	2	3	4	5	6	7	8	9	10	11	12
[R]	**[O]**	**[G]**	**[E]**	**[R]**	**[ ]**	[S]	[O]	[B]	[R]	[I]	[N]	[O]

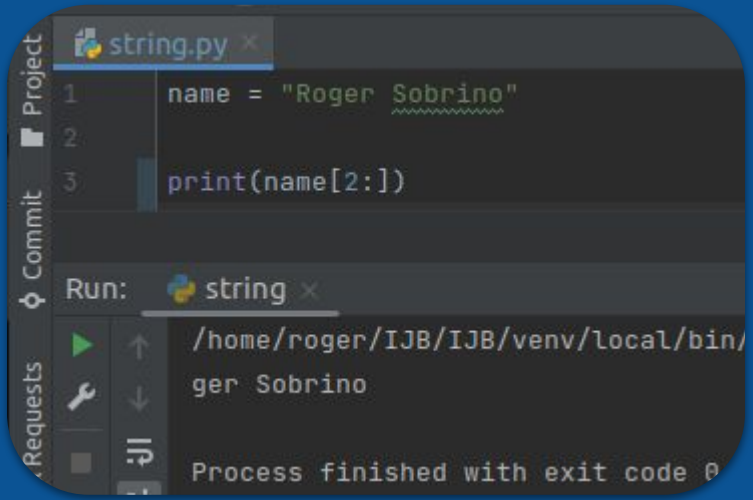
Data Types String

- Extreure caràcters d'un String



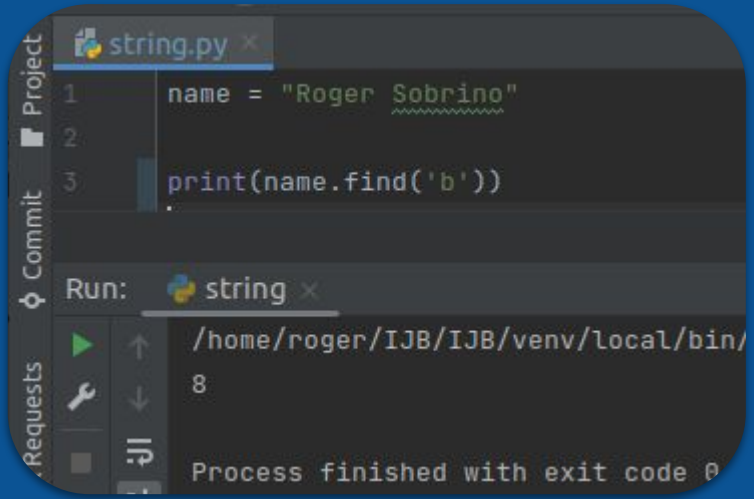
0	1	2	3	4	5	6	7	8	9	10	11	12
[R]	[O]	[G]	[E]	[R]	[ ]	[S]	[O]	[B]	[R]	[I]	[N]	[O]

- Extreure caràcters d'un String



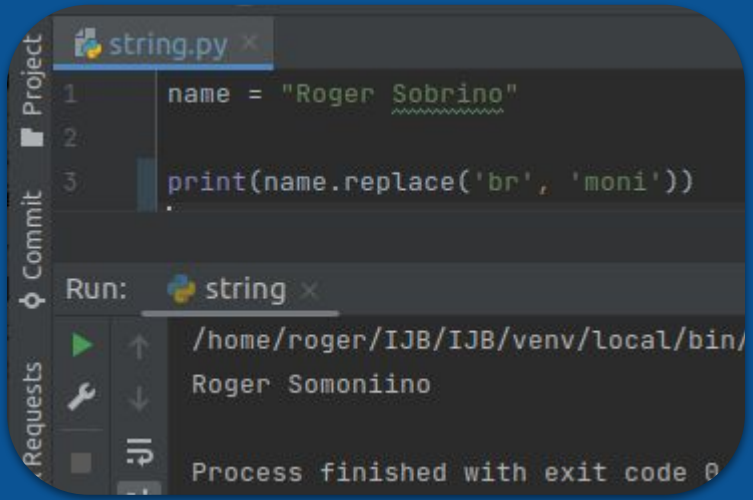
0	1	2	3	4	5	6	7	8	9	10	11	12
[R]	[O]	[G]	[E]	[R]	[ ]	[S]	[O]	[B]	[R]	[I]	[N]	[O]

- Trobar lletres dintre d'un String amb **find()**.



0	1	2	3	4	5	6	7	**8**	9	10	11	12
[R]	[O]	[G]	[E]	[R]	[ ]	[S]	[O]	**[B]**	[R]	[I]	[N]	[O]

- Canviar text d'un String amb **replace()**

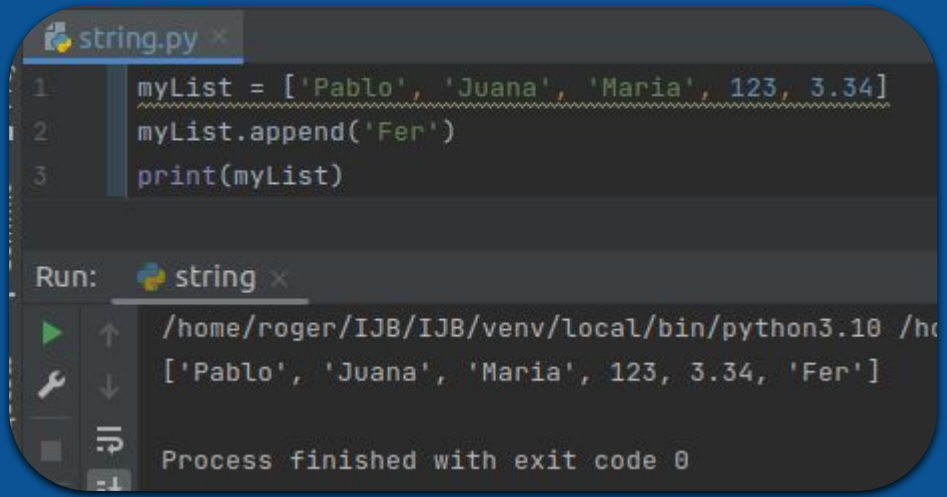


… i més mètodes ⇒

- Son col·leccions d'objectes de qualsevol tipus ordenades de forma arbitraria i sense tamany fixe.
- Son mutables (es poden modificar).
- Son una seqüència de valors indexats per números enters.

```
myList = ['Pablo', 'Juana', 'Maria', 123, 3.34]
```

- Afegir al final de la list amb el mètode **append()**.

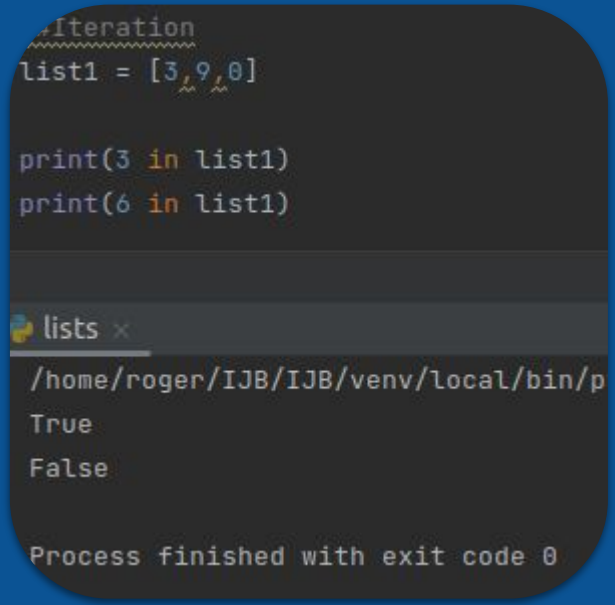


- Concatenar una o més lists amb **+**.

```
#Concatenació
list1 = [1,2,3]
list2 = [4,4,6]

print (list1 + list2)
print (list2 + list1)
```

```
lists
/home/roger/IJB/IJB/venv/local/bin/
[1, 2, 3, 4, 4, 6]
[4, 4, 6, 1, 2, 3]

Process finished with exit code 0
```

- Mirar si un valor està dintre d'una list (retorna un boolean) amb **in**.



- Quan volem veure un valor concret dintre d'una list ubicat en una posició (index) de la list (indexació).

```
##Indexing
list1 = [1, "hola", 3.90, [0,2,3], False]
print(list1[1])
print(list1[-3])
```

```
lists
/home/roger/IJB/IJB/venv/local/bin/python
hola
3.9

Process finished with exit code 0
```

- Les matrius son arrays multidimensionals. En aquest exemple tenim una **matriu 3x3** de lists.

```
matriu1 = [[1,2.3,"hola"],[3,True,0],["Roger", 18, False]]

print(matriu1[1])
print(matriu1[0][2])
print(matriu1[0][3])
```

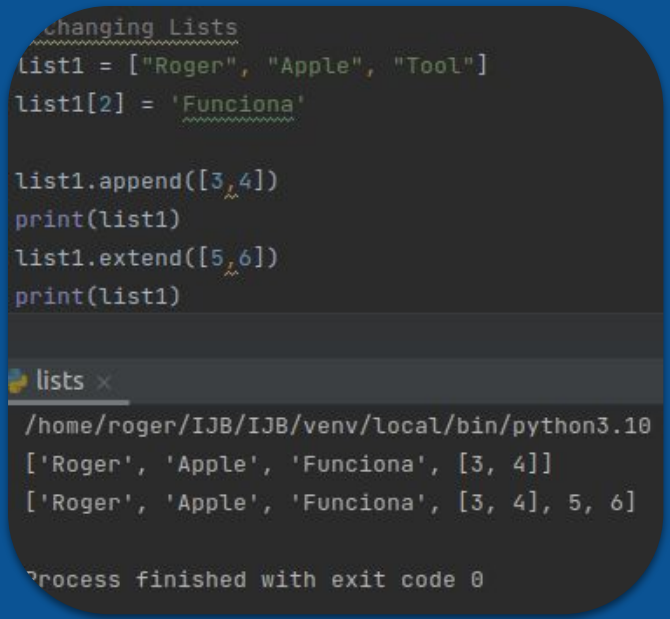
```
lists
/home/roger/IJB/IJB/venv/local/bin/python3.10 /home/roger/IJB/
[3, True, 0]
hola
Traceback (most recent call last):
  File "/home/roger/IJB/IJB/M7/Teoria/PYTHON/basics/lists.py"
    print(matriu1[0][3])
IndexError: list index out of range
```

- Dintre d'una list es poden modificar els seus valors.

```
#Changing Lists
list1 = ["Roger", "Apple", "Tool"]
list1[2] = 'Funciona'
print(list1)
```

```
lists
/home/roger/IJB/IJB/venv/local/bin/
['Roger', 'Apple', 'Funciona']

Process finished with exit code 0
```

- El mètode **append()** afegeix l'objecte list al final de la list
- El mètode **extend()** inserta els valors al final de la list.



- El mètode **sort()** serveix per ordenar de forma ascendent els valors de la list.

```
sort()
list1 = ["Roger", "Andreu", "Juan", "John", "Doe"]
list2 = [2,5,6,7,1,0,3,4]
list1.sort()
list2.sort()
print("sort de list1: ")
print(list1)
print("sort de list2: ")
print(list2)
```

```
lists
/home/roger/IJB/IJB/venv/local/bin/python3.10 /hom
sort de list1:
['Andreu', 'Doe', 'John', 'Juan', 'Roger']
sort de list2:
[0, 1, 2, 3, 4, 5, 6, 7]

ocess finished with exit code 0
```

- Elimina, de la list, l'últim valor amb el mètode **pop()**.

```
myList = [1,2,3,4]
print (myList.pop())
print(myList)
```

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/lists.py
4
[1, 2, 3]
```

Manera diferent de crear una list:

Codi:

```
numbers = [value**2 for value in range(1,11)]
print(numbers)
```

Resposta:

```
/home/roger/IJB/IJB/venv/bin/python /home/
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Tupla

- No Son mutables (no es poden modificar).
- Son una seqüència de valors indexats per números enters.
- Inicialització Tupla ⇒ **myTupla = ( )**

```
myTupla = (1,2,3,4,5,6,7)
```

- Mètode **len()** per conèixer la longitud de la Tupla.

```
myTupla = (1,2,3,4,5,6,7)
print(len(myTupla))
```

```
tupla
/home/roger/IJB/IJB/venv/bin/python
7

Process finished with exit code 0
```

- Per buscar un valor dintre d'una tupla s'utilitza **in**.

```
x = 9
y = 3
myTupla = (1,2,3,4,5,6,7)
print(_x in myTupla)
print(y in myTupla)
```

```
tupla
/home/roger/IJB/IJB/venv/bin/python /
False
True

Process finished with exit code 0
```

- Per saber quantes vegades es repeteix un valor dintre d'una tupla es fa servir el mètode **count( )**.

```
myTupla = (1,2,"pedro",3,4,"pedro",5,6,7)
print(myTupla.count("pedro"))
```

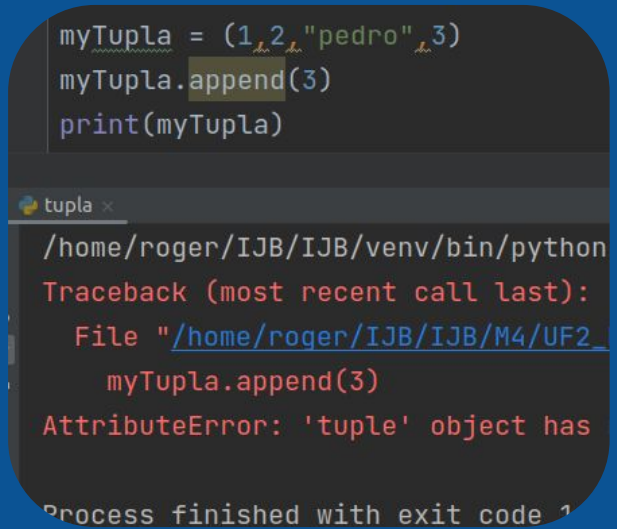
```
tupla
/home/roger/IJB/IJB/venv/bin/python /home/roge
2

Process finished with exit code 0
```

- Les tuples son immutables. No es poden modificar.

```
myTupla = (1,2,"pedro",3,4,"pedro",5,6,7)
myTupla[3] = "juan"
print(myTupla)
```

```
tupla
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py
Traceback (most recent call last):
  File "/home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py", line 58, in <module>
    myTupla[3] = "juan"
TypeError: 'tuple' object does not support item assignment
```

- Al ser, la tupla, inmutable, el mètode **append( )** no funciona.



- És una **col·lecció desordenada** sense elements duplicats.
- Suporten operacions matemàtiques com unió, intersecció, diferència i diferència simètrica.
- Els valors no es poden canviar, però es poden borrar i afegir.
- S'inicialitza amb ⇒ **mySet = set()**

```
mySet = set()
```

- Per buscar la diferència entre varis sets s'utilitza **-**.

```
x = set('a,b,c,d,e')
print(x)
y = set('y,b,z,d,j')
print(y)
print(y-x)
print(x-y)
```

```
set
/home/roger/IJB/IJB/venv/bin/python
{'c', 'a', 'b', 'e', ',', 'd'}
{'z', 'b', 'y', 'j', ',', 'd'}
{'y', 'j', 'z'}
{'a', 'e', 'c'}

Process finished with exit code 0
```

Interessant que mireu de donar-li al "run" varies vegades per veure com varia la resposta.

- Per unir varis sets s'utilitza |.

```
x = set('a,b,c,d,e')
y = set('y,b,z,d,j')
print(x | y)
```

```
set
/home/roger/IJB/IJB/venv/bin/python
{'a', 'j', 'd', 'z', 'c', 'b', 'y',

Process finished with exit code 0
```

- Per saber els valors repetits en varis sets s'utilitza la intersecció **&**.

```
x = set('a,b,c,d,e')
y = set('y,b,z,d,j')
print(x & y)
```

```
set
/home/roger/IJB/IJB/venv/bin/python
{'b', 'd', ','}

Process finished with exit code 0
```

- Per saber els valors no repetits en varis sets s'utilitza la **simetria (XOR)**.

```
x = set('a,b,c,d,e')
y = set('y,b,z,d,j')
print(x ^ y)
```

```
set
/home/roger/IJB/IJB/venv/bin/python
{'e', 'c', 'y', 'j', 'z', 'a'}

Process finished with exit code 0
```

- Per afegir valors a set s'utilitza **add()**.

```
x = set('a,b,c,d,e')
x.add('pedro')
print(x)
```

```
set
/home/roger/IJB/IJB/venv/bin/python
{'d', 'a', 'pedro', 'c', ',', 'e', '

Process finished with exit code 0
```

Interessant que mireu de donar-li al "run" varies vegades per veure com varia la resposta.

- Per borrar valors a set s'utilitza **remove()**.

```
x = set('a,b,c,d,e')
x.remove('c')
print(x)
```

```
set
/home/roger/IJB/IJB/venv/bin/python
{'d', ',', 'b', 'a', 'e'}

Process finished with exit code 0
```

- És un set desordenat que conté parells de “key:value”.
- El contingut està indexat per “key”.
- S’inicialitza amb ⇒ **myDict= {}**

```
myDict = {}
```

- Per afegir una key de nom **prova** i el seu valor:

```
myDict = {}
myDict["prova"] = 18
print(myDict)
```

```
dictionaries
/home/roger/IJB/IJB/venv/bin/pytho
{'prova': 18}

Process finished with exit code 0
```

- Per saber el tamany del diccionari s'utilitza **len()**:

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}
print(len(myDict))
```

```
dictionaries
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/
3

Process finished with exit code 0
```

- Per eliminar l'últim ítem del diccionari s'utilitza **popitem()**.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}
print(myDict.popitem())
print(myDict)
```

```
dictionaries
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/
('Edat', 38)
{'prova': 18, 'Roger': 'Sobrino'}

Process finished with exit code 0
```

- Per eliminar contingut amb coincidència de la key:

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}
del myDict['Edat']
print(myDict)
```

```
dictionaries
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/
{'prova': 18, 'Roger': 'Sobrino'}

Process finished with exit code 0
```

- Per saber tots els valors d'un diccionari s'utilitza **values()**.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}
print(myDict.values())
```

```
dictionaries
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB
dict_values([18, 'Sobrino', 38])

Process finished with exit code 0
```

MOLTES GRÀCIES!